

Data Structures Using C

Course Code: CSIT124

B. Tech. (CSE)

Module 1

Dr. Sheenu Rizvi

Asst. Professor

Complexity

Complexity of an algorithm is a measure of the amount of **time**(how many steps it takes to solve a problem) and/or **space** (how much memory does it take to solve a problem) required by an algorithm for an input of a given size.

What effects run time of an algorithm?

- (a) computer used, the hardware platform
- (b) representation of abstract data types (ADT's)
- (c) efficiency of compiler
- (d) competence of implementer (programming skills)
- (e) complexity of underlying algorithm
- (f) size of the input

Example.

if we are analysing a **sorting algorithm** we might count the number of comparisons performed, and if it is an algorithm to find some optimal solution, the number of times it evaluates a solution.

We take three cases for complexity of algorithm.

Best case : Minimum time required for program execution.

Worst case: Maximum time required for program execution.

Average case: Average time required for program execution.

Big O notation

A notation has been developed to facilitate the handling of order of magnitude functions. A function $f(n)$ is defined to be $O(g(n))$, i.e. $f(n) = O(g(n))$ and is said to be of order $g(n)$ if there exist +ve constants n_0 and C such that

$$|f(n)| \leq C * |g(n)| \text{ for all } n \geq n_0$$

The notation $f(n) = O(g(n))$ (read as f of n equals big-oh of g of n) has a precise mathematical definition.

$f(n)$ will normally represent the computing time of some algorithm, when we say that computing time of an algorithm is $O(g(n))$, we mean that its execution take no more than a const time $g(n)$, n is parameter which characterises the input and the output.

We write $O(1)$ to mean a computing time which is constant. $O(n)$ is called linear, $O(n^2)$ is called quadratic, $O(n^3)$ is called cubic and $O(2^n)$ is called exponential.

If an algorithm takes time $O(\log n)$ it is faster, for sufficiently large n , than if it had taken $O(n)$. Similarly, $O(n \log n)$ is better than $O(n^2)$ but not as good as $O(n)$.

If we have two algorithm perform same task and first have computing time which is $O(n)$ and the second $O(n^2)$ then we will usually take first as superior, the reason is that as 'n' increases the time for the second algorithm will get far worse than the time for the first.

In a fig. below show that how the computing time (counts) grow with a const equal to one, how the times $O(n)$ and $O(n \log_2 n)$ grow much more slowly than other.

n	$\log_2 n$	$n \log_2 n$	n^2	n^3	2^n
1	0	0	1	1	2
2	1	2	4	8	4
4	2	8	16	64	16
8	3	24	64	512	256
16	4	64	256	4096	65536

Values for computing functions

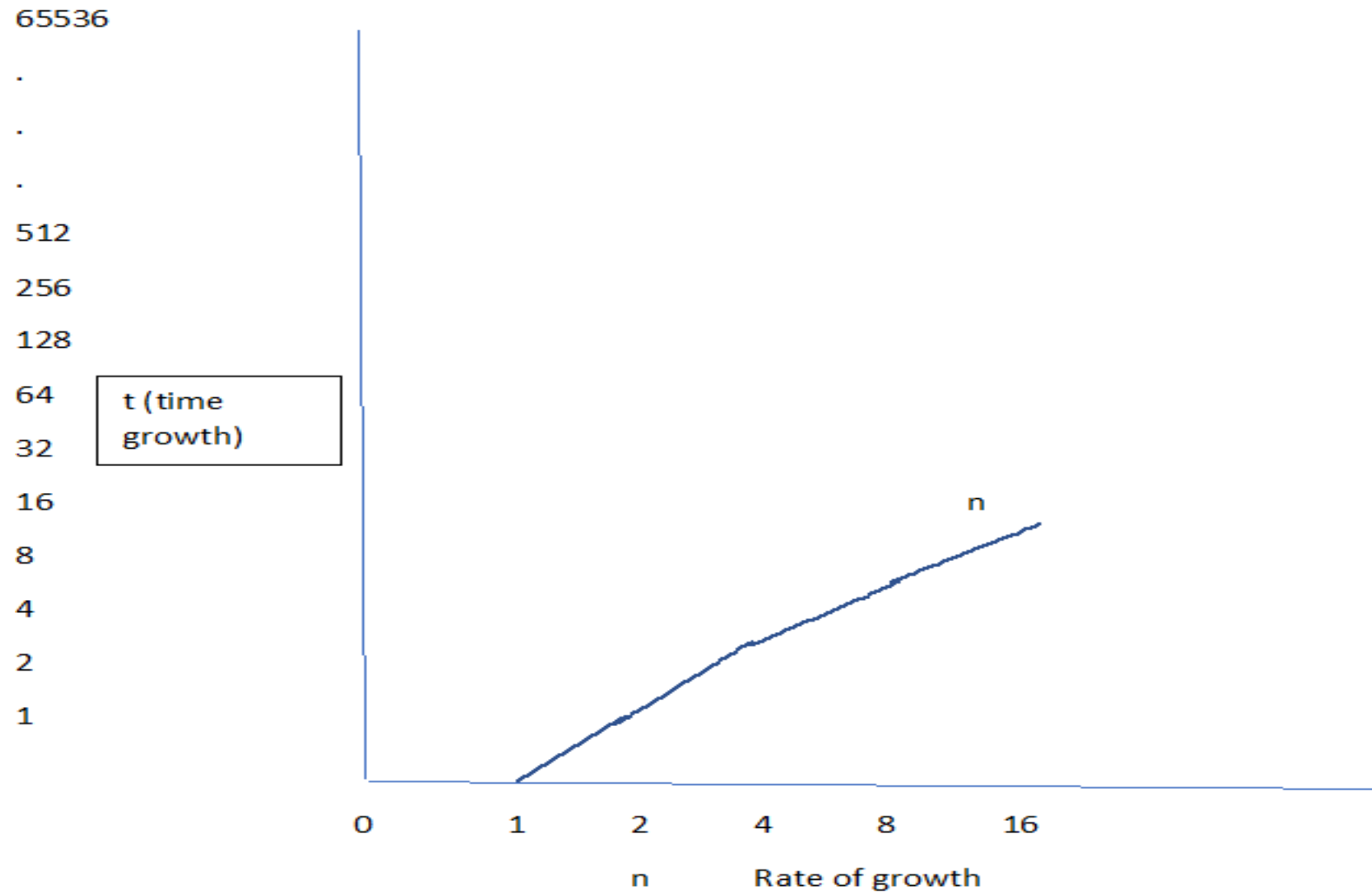


figure for $n \log_2 n$

Let us take some situations and behaviour of algorithm in order.

$O(1)$: When we get data in first time itself.

Ex. Hash table

$O(n^2)$: When the full list will be traversed for each element.

Ex. worst case of bubble sort.

$O(\log n)$: When we divide list half each time and traverse the middle element

Ex. Binary searching, binary tree traversal

$O(n \log n)$: When we divide list half each time and traverse that half portion.

Ex Best case of quick sort.

Pointer

A pointer is a variable it contains memory address(locations) of another variable which is declare as the same data type.

Ex `int i = 3;`

The declarations tell the compiler

- a) Reserves space in memory to hold integer value.
- b) The name i with this memory location.
- c) Store the value 3 at this location

i location name

3

 Value at location

6232 location number (address)

Syntax

Data type *pointer name;

Here, * before pointer indicate the compiler that variable declared as a pointer.

Example:

```
int *p1; //pointer to integer type
```

```
float *p2; //pointer to float type
```

```
char *p3; //pointer to character type
```

When pointer declared, it contains garbage value i.e. it may point any value in the memory.

Two operators are used in the pointer i.e. address operator(&) and indirection operator or dereference operator (*).

Example:

```
void main()

{

    int i=105;

    int *p;

    p=&i;

    printf("value of i=%d",*p);

    printf("value of i=%d",&i);

    printf("address of p=%u",&p);

}
```

//input a string and find length of string using pointer

```
#include<stdio.h>
```

```
void main()
```

```
{
```

```
char str[10];
```

```
int i=0;
```

```
char*ptr;
```

```
ptr=str;
```

```
printf("enter a string");
```

```
gets(str);
```

```
while(*ptr!='\0')
```

```
{ptr++;
```

```
i++;
```

```
}
```

```
printf("length of string is %d",i);
```

```
}
```

WAP input string and remove trailing spaces using pointer

```
#include<stdio.h>

void main()
{   char str[10];
    char*p;
    int i=0;
    p=str;
    puts("enter string");
    gets(str);
    while(*p!='\0')
    {
        p++;
    }
```

```
p--; //REMOVE '\0'
```

```
while(*p==' ')
```

```
{p=p-1;
```

```
i++;
```

```
}
```

```
p=p+1; //INCREMENT TO WRITEEND OF STRING '\0'
```

```
*p='\0';
```

```
printf("%s\n",str);
```

```
printf("blank =%d",i);
```

```
}
```

WAP swapping of two number

```
#include<stdio.h>
```

```
#include<conio.h>
```

```
void swap(int*a, int*b);
```

```
void main()
```

```
{ int a=10;
```

```
int b=20;
```

```
swap(&a,&b);
```

```
printf("\na=%d\n",a);
```

```
printf("b=%d",b);
```

```
getch();
```

```
}
```

```
void swap(int *x,int *y)
{
int t;
t=*x;
*x=*y;
*y=t;
}
```


Use of pointer

Pointer are used for saving memory space.

- 1) The use of pointer assigns the memory space and releases it. This concept help in making the best use of the available memory (dynamic memory allocation)
- 2) With pointer data manipulation is done with address so the execution time is faster.
- 3) Concept of pointer used in data structure such as linked list.

Thank You